

HW 2

Wednesday, October 23, 2024 9:27 PM

1) Let the function fun be defined as

```
int fun(int* k) {
    *k += 4;
    return 3 * (*k) - 1;
}
```

Suppose fun is used in a program as follows:

```
void main() {
    int i = 10, j = 10, sum1, sum2;
    sum1 = (i / 2) + fun(&i);
    sum2 = fun(&j) + (j / 2);
}
```

What are the values of sum1 and sum2

1. operands in the expressions are evaluated left to right? (10 points)
2. operands in the expressions are evaluated right to left? (10 points)

$$1.) \text{sum1} = \frac{10}{2} + (10+4)3 - 1$$

$$5 + (14)3 - 1$$

$$41$$

$$46$$

$$2.) \text{sum1} = (10+4)3 - 1 + \frac{14}{2}$$

$$42 - 1 + 7$$

$$41 + 7$$

$$48$$

$$\text{sum2} = (10+4)3 - 1 + \frac{14}{2}$$

$$(14)3 - 1 + 7$$

$$42 - 1 + 7$$

$$41 + \frac{14}{2}$$

$$48$$

$$\text{sum2} = \frac{10}{2} + (10+4)3 - 1$$

$$5 + 41$$

$$46$$

2) Consider the following program written in C syntax:

```
void swap(int a, int b) {
    int temp;
    temp = a;
    a = b;
    b = temp;
}

void main() { int value = 2, list[5] = {1, 3, 5, 7, 9};
    swap(value, list[0]);
    swap(list[0], list[1]);
    swap(value, list[value]);
}
```

For each of the following parameter-passing methods, what are all of the values of the variables value and list after each of the three calls to swap?

1. Passed by value (20 points)
2. Passed by reference (20 points)
3. Passed by value-result (20 points)

1. value = 2 list = [1, 3, 5, 7, 9]
 value = 2 list = [1, 3, 5, 7, 9]
 value = 2 list = [1, 3, 5, 7, 9]

2. value = 1 list = [2, 3, , 7, 9]
 value = 1 list = [3, 2, 5, 7, 9]
 value = 2 list = [3, 1, 5, 7, 9]

3. value = 1 list = [2, 3, 5, 7, 9]
 value = 1 list = [3, 2, 5, 7, 9]
 value = 2 list = [3, 1, 5, 7, 9]

3) What are the arguments both for and against the exclusive use of Boolean expressions in the control statements in Java (as opposed to also allowing arithmetic expressions, as in C++)? (10 points)

For -

Boolean expressions can reduce the risk of logical errors brought on by potential syntax errors, also reducing the amount of ambiguity in the code and increasing the overall consistency and readability. This makes it easier for compilers to identify potential issues and optimize the code.

Against -

Explicitly using Boolean expressions can induce limitations into certain statements. This makes it hard to address numeric values that have implicit meanings, lowering the overall flexibility of the code. The conciseness of the program is impacted well due to Boolean expressions requiring explicit comparison

4) What are at least two arguments against the use of pass-by-name parameters? (10 points)

Pass by name parameters do not follow a single implementation model, which requires a subprogram to evaluate the address/value of the parameter. The referencing environment of the sub program must also be passed.

Pass by name parameters are complex to implement, they have inefficiencies that lower a programs readability and reliability